VIT

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science Engineering and Information Systems

Fall Semester 2026-2027

BCSE303P

Operating Systems Lab

PROJECT REVIEW - 1

TITLE

SMART HOSPITAL RESOURCE MANAGEMENT SYSTEM

Name(s) (Reg no)

Havish (24BYB0057)

Shanmukha Simha (24BDE0123)

Date of Submission: 04 September 2026

Under the Guidance of

Dr. PORKODI S

Assistant Professor Senior Grade – 1

School of Computer Science Engineering and Information Systems

ABSTRACT: 150 – 200 WORDS

In this Operating Systems lab mini project, we developed a Smart Hospital Resource Management System to apply core OS theoretical concepts to a practical real-world scenario. In emergency hospital wards, specialized medical units such as ICU beds, mechanical ventilators, MRI scanners, and surgery rooms are limited in number. When multiple patients arrive simultaneously with different urgency levels, managing patient queues and equipment allocation manually leads to long waiting times, equipment booking conflicts, and potential surgical deadlocks. Rather than implementing simple standalone textbook lab programs, our project models hospital emergency operations by integrating five core Operating System concepts into a single working web simulation framework. The system implements Preemptive Priority CPU Scheduling with aging for patient triage, Process Control Blocks (PCB) tracking 5-state process transitions, Mutex Locks and Semaphores to prevent race conditions on shared medical units, Banker's Safety Algorithm and Resource Allocation Graphs (RAG) to prevent surgical deadlocks, and Main Memory Partitioning with LRU/FIFO Virtual Page Replacement for diagnostic files. The simulator provides a live visual dashboard where scheduling performance and safety states can be evaluated interactively.

KEYWORDS: 5 – 6 KEYWORDS

CPU Scheduling, Process Control Block (PCB), Mutex Synchronization, Banker's Algorithm, Resource Allocation Graph (RAG), Virtual Memory Paging.

INTRODUCTION:

Operating System principles form the foundation for managing shared hardware resources efficiently among competing tasks. In our lab curriculum, topics such as process state management, CPU scheduling algorithms, mutual exclusion locks, deadlock avoidance, and memory management are typically studied through isolated CLI programs. For our mini project under the guidance of Dr. Porkodi S., we aimed to demonstrate how these individual OS concepts can be combined to solve a real-world resource allocation problem.

We selected hospital emergency room operations as our application domain. A modern hospital emergency ward operates under conditions identical to a multi-programmed operating system: doctor consultation time represents CPU core execution, patient registration represents process creation, medical equipment represents shared critical sections, and patient medical files represent memory allocations. Building this system helped us understand the practical behavior of scheduling algorithms and concurrency controls in a clear, visual manner.

PROBLEM STATEMENT:

In hospital emergency wards, uncoordinated patient processing and unmanaged equipment requests lead to critical operational bottlenecks:

- Emergency trauma cases waiting behind routine checkup cases due to lack of preemptive priority queueing.
- Lower-priority patients suffering from indefinite queue starvation during high emergency influxes.

- Race conditions occurring when multiple medical teams attempt to book the same single ICU bed or MRI scanner simultaneously.
- Circular wait deadlocks occurring during complex operations requiring multiple interdependent medical machines.
- Inefficient handling and page fault delays when accessing large patient digital diagnostic imaging files in memory.

OBJECTIVES:

- To model patient admissions as OS Processes containing a complete Process Control Block (PCB) with 5-state lifecycle transitions.
- To implement Preemptive Priority Scheduling for emergency triage along with an Aging algorithm to prevent starvation of low-priority routine cases.
- To protect shared physical equipment (ICU, Ventilator, MRI, Surgery Room) using Mutex Locks and Semaphores to guarantee mutual exclusion.
- To apply Banker's Safety Algorithm and canvas-based Resource Allocation Graphs (RAG) with cycle detection to avoid surgical deadlocks.
- To manage main RAM partitioning (First-Fit, Best-Fit, Worst-Fit) and simulate Virtual Memory Paging using LRU and FIFO page replacement.

SCOPE OF THE PROJECT:

The scope of this mini project includes the design and implementation of a web-based simulation environment that models hospital triage and equipment management. The system allows users to dynamically add patient processes with varying priorities (P1 Emergency Trauma to P4 Routine Checkup), CPU burst times, RAM requirements, and equipment claims. Users can switch between different CPU scheduling algorithms (Preemptive Priority, Round Robin Q=2s, SJF, FCFS) live, observe preemption events, run Banker's safety state evaluations, monitor Mutex lock waiting queues, test page replacement hit/fault ratios, and inspect comparative performance metrics (Average Wait Time, Turnaround Time, CPU Throughput).

PROPOSED SYSTEM:

The proposed system integrates five core Operating System concepts into a unified execution engine:

- **1. CPU & Priority Scheduling:** The triage doctor CPU core schedules patients based on clinical priority (P1 to P4). When a P1 Emergency Trauma case enters the queue, any lower-priority process currently executing is preempted and returned to the READY state. Aging boosts effective priority for long-waiting P4 cases.
- **2. Process Control (PCB):** Each patient case is stored as a PCB object maintaining PID, Name, Priority, Burst Time, Memory Required, Resource Claim, State (NEW, READY, RUNNING, WAITING, TERMINATED), and accounting timestamps.
- **3. Thread Synchronization:** Shared units (ICU Bed, Ventilator, MRI, Surgery Room) are protected by Mutex locks. Requesting an occupied resource moves the patient process to a WAITING queue, avoiding race conditions.

- **4. Deadlock Management:** Banker's Algorithm evaluates `Available`, `Max`, `Allocation`, and `Need` matrices before granting claims to guarantee a Safe Sequence. An interactive RAG graph visualizes process/resource nodes and detects cycle deadlocks.
- **5. Memory & Paging:** 1024 KB RAM is managed using First-Fit, Best-Fit, and Worst-Fit dynamic partitioning. Virtual Paging models 4 physical frames tracking Page Hits and Faults under LRU and FIFO replacement policies.

SYSTEM ARCHITECTURE:

The application follows a modular, client-side software architecture consisting of a presentation frontend, five specialized OS simulation engine modules, and a central simulation loop ticker:

- **Frontend Layer (`index.html`, `styles.css`):** Responsive single-page glassmorphism dashboard containing controls, 5 concept tab viewports, data tables, Gantt chart canvas, RAG canvas, and console logs.
- **Process Manager (`process_manager.js`):** Handles PCB creation, process registry, PID generation, and state transitions.
- **CPU Scheduler Engine (`cpu_scheduler.js`):** Controls Priority Preemptive, RR, SJF, and FCFS algorithms, aging logic, context switching, and Gantt timeline generation.
- **Banker & RAG Module (`banker_rag.js`):** Solves safety matrices, computes safe sequences, and draws RAG nodes and edges on canvas.
- **Synchronization Engine (`synchronization.js`):** Manages Mutex locks and critical section waiting queues.
- **Memory & Paging Manager (`memory_manager.js`):** Manages First/Best-Fit RAM partitioning and LRU/FIFO virtual page replacement.
- **Central OS Engine & UI Controller (`os_engine.js`, `ui_controller.js`):** Runs the clock ticker (0.8s/tick), handles DOM events, and renders UI state.

REFERENCES: (10 REFERENCE)

1. A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.
2. W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Boston, MA, USA: Pearson, 2018.
3. A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA, USA: Pearson, 2015.
4. E. W. Dijkstra, "Cooperating sequential processes," in *Programming Languages*, F. Genuys, Ed. New York, NY, USA: Academic Press, 1968, pp. 43–112.
5. E. G. Coffman, M. J. Elphick, and A. Shoshani, "System deadlocks," *ACM Computing Surveys*, vol. 3, no. 2, pp. 67–78, Jun. 1971.
6. R. M. Fujimoto, "Parallel and distributed simulation systems," *Proceedings of the 33rd conference on Winter simulation*, IEEE, 2001, pp. 147–157.
7. L. Kleinrock, *Queueing Systems, Volume 1: Theory*. New York, NY, USA: Wiley-Interscience, 1975.
8. H. M. Deitel, P. J. Deitel, and D. R. Choffnes, *Operating Systems*, 3rd ed. Upper Saddle River, NJ, USA: Pearson/Prentice Hall, 2004.
9. M. L. Pinedo, *Scheduling: Theory, Algorithms, and Systems*, 5th ed. New York, NY, USA: Springer, 2016.

10. IEEE Computer Society, "IEEE Standard for Information Technology—Portable Operating System Interface (POSIX) Base Specifications," *IEEE Std 1003.1-2017*, 2018.